

docs-chain

Docs Chain in this repository

Revision: 1 **Last modified:** 2026-09-03T00:00:00Z **Status:** ACTIVE — the umbrella’s own Markdown is bound; the fleet determination below is deliberate and states what is NOT bound and why. **Authority:** constitution §11.4.106 (Docs Chain is the canonical sync engine), §11.4.65 (the export mandate), §11.4.28(B) (submodule decoupling)

1. What is mandated, and by which rule

The format set is `.html` + `.pdf`. The citation is constitution §11.4.65 — **Universal Markdown export mandate**, whose verbatim operator anchor reads:

“Any markdown document inside the project and which is not part of the applications or services source code MUST BE exported (be available) in PDF and HTML! ... Every time when Markdown document (file) is modified, its proper HTML and PDF versions MUST BE regenerated. Markdown documents MUST BE at all times in sync with PDF and HTML versions!”

and whose mandatory protection 1 is “*Every INCLUDED .md file has .html and .pdf siblings.*”

DOCX is NOT in the universal set. It is added by §11.4.153 for one document class only — the per-feature Status set — which says so explicitly: “*this doc class ADDS DOCX to the §11.4.65 universal HTML+PDF export set; the other doc classes are unchanged.*” §11.4.208 (operator-request history) and the extension-catalogue rule likewise name a four-format export for their own classes. **This repository**

currently has no document of any of those classes, so no chain here emits DOCX. Demanding one would be a §11.4.201(1) false-positive refusal — a gate failing a tree that is compliant.

Nothing was recorded as *could not determine* on the format axis: every mandated format is producible on this host, measured 2026-09-03 — pandoc 3.10, weasyprint 69.0, and (for the DOCX class if one ever lands) pandoc's own pandoc-docx builtin plus libreoffice. Docs Chain surfaces an absent tool as a typed `ToolAbsentError` and an honest SKIP rather than a fake transform (§11.4.106(E)); that path was not exercised because no tool is absent.

2. How Docs Chain is adopted — through the constitution, not a second copy

`docs_chain` is **already a submodule of the constitution**, at `submodules/constitution/submodules/docs_chain`, landed there under the §11.4.28(C) depth-1 reusable-engine carve-out. **No gitlink was added to this repository**, because adding one would create a second checkout of `vasic-digital/docs_chain` in the same tree — precisely the duplication §11.4.74 (reuse before reimplement) exists to prevent, and it would put two independently-movable pins on one upstream.

Consequences of consuming it this way, stated rather than implied:

- `.gitmodules` and `helix-deps.yaml` are **unchanged**, so cascade check C9 (which compares the manifest to the git **index**, `git ls-files -s`) cannot drift on account of this work. Nothing needed to move together because nothing moved.
- The engine is consumed **by reference**, never copied (§11.4.106(A), §11.4.177). This repository holds configuration and a gate; it holds no vendored copy of the engine.
- The binary is a build artefact, resolved at run time in this order:
`$DOCS_CHAIN_BIN` → `PATH` → the module's own gitignored build output. Build it with:

```
(cd submodules/constitution/submodules/docs_chain && go build  
-o docs_chain ./cmd/docs_chain)
```

If none of the three resolves, `scripts/verify-docs-chain.sh` exits 2 (could not determine) and prints that command. An absent tool is never reported as a clean tree — a missing binary and a compliant repository would otherwise return the same quiet zero (§11.4.201(6)).

3. What is bound

Three contexts under `.docs_chain/contexts/`, all **generated** by `scripts/gen-docs-chain-contexts.sh`:

Context	Scope	Documents
umbrella-root	project-root *.md — README, CONTINUATION, Constitution.md, and the four carriers	7
umbrella-docs	docs/**/*.md	51
umbrella-specs	specs/**/*.md	54

The 40 / 25 figures this table carried are SUPERSEDED, re-measured 2026-09-09 at 51 and 54 — a roster of 112 documents, not 72. Nothing regressed: the generator re-derived the roster from a tree that grew, which is C3 working rather than drift. Read the consequence rather than the counts: C5’s “artifacts absent” figure rose from 144 to 206 across the same move, and that rise is arithmetic, not a new defect — $75 \text{ documents} \times 2 \text{ slots} = 150$ became $112 \times 2 = 224$, so the DENOMINATOR became honest while the numerator followed. Re-derive rather than quoting this table:

```
bash scripts/gen-docs-chain-contexts.sh # (re)derive
grep -c 'markdown' .docs_chain/contexts/*.yaml
```

Each document is one two-edge chain: `md` → `html` (`pandoc-html`) then `html` → `pdf` (`weasyprint-pdf`). The PDF is rendered from the **same HTML a reader opens**, never from the Markdown directly, so the two exports cannot become different renderings of one source.

Excluded, each as a §11.4.65 EXCLUDED class rather than a convenience: `.claude/`, `.specify/`, `.ashlrcode/`, `.github/` (vendored agent, plugin and extension trees this repository does not own), dependency trees, and **every path declared in .gitmodules** — see §4.

The roster is derived, and its rot is a failure

Docs Chain binds explicit node paths; its schema has no glob for a markdown node (`members: exists only for kind: fingerprint`). A hand-written context is therefore a roster, and a roster stops matching reality the day someone adds a document and does not remember the file exists.

So the roster is **generated from the tree**, and `scripts/verify-docs-chain.sh` check **C3** re-derives it into a temp directory and diffs. **A Markdown document added later with no node is a diff, which is a FAILURE** — not an omission. That pairing, not the generator alone, is what makes “and for documents added later” true rather than aspirational. `docs_chain verify` cannot see this: it reports on what is *registered* and is perfectly content with a silently shrinking scope.

4. The privacy rule, and how the configuration enforces it

This repository is **PUBLIC**. Three of its submodules — `workshop`, `ai_interviewing`, `monetization` — are **PRIVATE**. An HTML or PDF export carries the **full text** of its source. Docs Chain node paths are project-root-relative, so a context at *this* root could name `workshop/docs/x.md` and emit `workshop/docs/x.html` — or, with one wrong path, emit that HTML at a public path. Committed and pushed, that is a permanent, irreversible disclosure: history is not editable after a push.

The defence is structural, not careful. Three layers:

1. **Scope.** Every path in every context at this root is a file *this* repository tracks. **No submodule path — public or private — is ever named.** A private source therefore has no node in any public-root chain, so no transform can be asked to render one. There is no path from a misconfiguration of this repository’s chains to a public export of a private document, because the configuration does not name private documents at all.
2. **Mechanical guard.** `scripts/verify-docs-chain.sh` asserts it on every run, and fails (exit 1) rather than warns:
 - **C1 CONTAINMENT** — no node path may name a **declared submodule**. The roster comes from `.gitmodules` at run time, so the rule is decidable without consulting any remote’s visibility setting — a property this tree cannot see and which changes

outside it. It refuses public submodules too: a rule that needed to know which submodules are private would be wrong the moment a visibility flag flips.

- **C2 NO ESCAPE** — no absolute path, no `..` segment. C1 closes the route into a submodule; C2 closes the route out of the repository entirely.

3. Per-repository chains. A submodule that wants a chain registers it in **its own root**, where its exports land in **its own repository** by construction (§11.4.28(B): a submodule is project-not-aware and self-contained). `docs_chain --root DIR` supports exactly this.

Both invariants are *demonstrated*, not asserted: `bash scripts/verify-docs-chain.sh --prove-failure` seeds a node under a declared submodule path (**M4**) and a `../` escape (**M5**) into a throwaway git repository and requires exit 1 from each, alongside a golden-FALSE control that must **not** fire.

5. The fleet determination — which owned submodules are bound

The fleet is derived at run time from `.gitmodules + helix-deps.yaml`, never from a list in a document. As measured: **13 declared gitlinks — 11 owned consumers, 1 governance source (submodules/constitution), 1 third-party (submodules/superspec).**

Determination: none of the 13 is bound from this root, and that is the correct answer rather than a shortfall.

Submodule class	Binding	Reason
The 3 private consumers	in its own root, by its own operator	binding them from a PUBLIC root is the exact leak class §4 exists to prevent §11.4.28(B) — a submodule’s export chain is part of that submodule, not of whatever project consumes it; binding from here would write into another repository and make its gitlink
The 8 public owned consumers	in its own root	

Submodule class	Binding	Reason
submodules/ constitution	already self-bound	dirty on every umbrella sync it is the governance SOURCE, and it ships CLAUDE.html/.pdf/.docx, AGENTS.*, Constitution.* beside their .md today. A second consumer writing into it would be a §11.4.26 violation third-party, and §11.4.65 EXCLUDES “any third-party submodule NOT in the owned-submodule set”
submodules/ superspec	not bound	

scripts/gen-docs-chain-contexts.sh is repository-agnostic and takes --root, so a submodule adopts its own chain with the same two commands used here. That is the mechanism; running it inside eleven separately-versioned repositories, several under concurrent edit, is an operator decision and is **recorded as an open item, not silently done**.

6. Running it

```

bash scripts/gen-docs-chain-contexts.sh           # (re)derive the
           contexts from the tree
DC=submodules/constitution/submodules/docs_chain/docs_chain
$DC doctor --all                                  # validate
           configuration, no writes
$DC sync    --all                                 # propagate;
           atomic, content-hash gated
$DC verify  --all                                  # read-only drift check
bash scripts/verify-docs-chain.sh                 # the gate:
           contained + complete + in sync

```

scripts/verify-docs-chain.sh is three-valued — **0** contained/complete/ in sync, **1** a real finding (drift, an unbound document, a path that leaves this repository), **2** could not determine (no root, not a git repository, no binary, a context that will not parse). **2 is never a pass**, and a confirmed failure outranks an undetermined. It is registered in

scripts/check-registry.tsv as docs-chain-sync with a --prove-failure paired proof, so R5 (anti-drift) is satisfied and the check is enumerated for SC-012/SC-013.

7. Honest boundaries

- **Content-hash, not mtime.** touch on a chained source triggers nothing — demonstrated, not asserted (§8 of the session report). This also means a §11.4.65 protection-2 reading of “mtime ≥” is satisfied *a fortiori*: the content is proven equal, which is stronger than a timestamp comparison.
- **The gate proves SYNC, not CORRECTNESS.** An export that faithfully renders a wrong document passes. Document correctness stays §11.4.44 / §11.4.99 / §11.4.194 territory.
- **REVERSED 2026-09-09, on an explicit operator decision: the exports ARE TRACKED.** The bullet below is kept verbatim, not deleted, because its measurement was real and its trade is real — it was decided the other way. What the old reasoning did not weigh is that ignoring the exports made §11.4.65 compliance a **working-tree property only**: a fresh clone carries no exports, so scripts/verify-docs-chain.sh — which checks protections 1 and 2 ON DISK — is RED on arrival for anyone who has not first run a sync, and §11.4.65’s own word “available” cannot mean “available to whoever generated it locally”. The §11.4.30 objection is answered narrowly rather than waved away: §11.4.30 forbids versioned BUILD artefacts of the applications and services; a rendering another anchor mandates must EXIST is the deliverable, not its by-product. The content-boundary cost is **accepted and expected** — that gate’s count rises, and a rise caused by tracking renderings of already-public Markdown is inherent to the mandate, **not a new disclosure**. The reversal is a NARROWING of .gitignore, not a blanket un-ignore: root *.pdf/*.png/*.html stay ignored by default with only the seven umbrella-root siblings negated by name, and / qa-results/docs_chain/ plus .docs_chain/state.json are untouched. .gitignore carries the full record. The paragraph that follows is the SUPERSEDED 2026-09-03 decision:
- **The exports are generated but NOT tracked, and that was decided on a measurement rather than a preference.** (*SUPERSEDED — see the bullet above.*) The content-boundary gate

reads a text-bearing PDF through `pdftotext` and HTML as text, so every export re-presents its source's full text to the detector. Measured on this tree, 2026-09-03:

corpus	result
tracked only (the gate's default)	LEAK – 11973 match(es)
plus the 146 exports (-- include-untracked)	LEAK – 24759 match(es)

+12,786 matches, +107% — on a gate that is red by design and whose whole value is that a human still reads it. Tracking the exports would more than double it while disclosing nothing new: every export renders a `.md` that is already public. §11.4.30 forbids versioning a build derivative, and §11.4.77 is satisfied because the regeneration mechanism exists and is gated (`docs_chain sync --all`). So `.gitignore` now covers the exports, which also resolves — coherently — the pre-existing asymmetry in which the root `/*.pdf` rule ignored the root PDFs while the root `.html` would have been tracked.

This does **not** weaken §11.4.65: protections 1 and 2 are about the siblings EXISTING and being IN SYNC, and `scripts/verify-docs-chain.sh` checks that **on disk** (C5), never in the index — so ignoring them costs the gate not one assertion. It is a reversible decision: delete one commented block in `.gitignore` and the exports become tracked. The fleet precedent points the other way — the constitution's own repository tracks its `.html/.pdf/.docx` — so an operator who prefers that can have it for the price of the measured match count above.

- **Not bound:** the eleven owned submodules (§5), and every vendored tree under `.claude/`, `.specify/` and `.ashlrcode/`. Both are declared scope boundaries, not claims of conformance.
- **C4 costs roughly what a sync costs, and that has a consequence.** `docs_chain verify` re-derives every registered output to compare it, so on this 73-document corpus C4 takes **6–10 minutes** on a loaded host (measured: a full `sync --all` was 29m25s at 26% CPU under a load average of 25). Wiring this gate into `scripts/pre-push-gates.sh` would add that to every push. The cheap pre-filter is the mtime comparison C5 already implies — if no `.md` is newer than its `.html`, C4 has nothing to find — and adding that fast path is a reasonable follow-up. **It was not done here**, because a check that skips its authoritative comparison on an mtime heuristic is

precisely the mtime-vs-content-hash confusion this engine exists to remove, and that trade-off deserves a deliberate decision rather than a quiet optimisation.

- **On a tree under concurrent edit, C4 can be red through nobody's fault.** Measured twice, 2026-09-03: a sync --all left **zero** drifted sources, and the C4 that followed still reported DRIFT because CONTINUATION.md was edited again *during* C4's own multi-minute run — md 20:35:12 against an export built 20:29:14, and before that md 20:16:41 against 20:13:53. The offending document is the one several agents update continuously. That is the gate reporting a genuinely stale export, which is what it is for; it is also why a green C4 should be read as “green at the instant it finished”, and why this gate belongs at a quiescent seam (pre-push, per §11.4.84) rather than mid-flight.